scientific reports

Check for updates

OPEN

Few-shot traffic classification based on autoencoder and deep graph convolutional networks

Shengwei Xu[1], Jijie Han[2]✉, Yilong Liu[2,3], Haoran Liu[2] & Yijie Bai[2]

Traffic classification is a crucial technique in network management that aims to identify and manage data packets to optimize network efficiency, ensure quality of service, enhance network security, and implement policy management. As graph convolutional networks (GCNs) take into account not only the features of the data itself, but also the relationships among sets of data during classification. Many researchers have proposed their own traffic classification methods based on GCN in recent years. However, most of the current approaches use two-layer GCN primarily due to the over-smoothing problem associated with deeper GCN. In scenarios with small samples, a two-layer GCN may not adequately capture relationships among traffic data, leading to limited classification performance. Additionally, during graph construction, traffic usually needs to be trimmed to a uniform length, and for traffic with insufficient length, zero-padding is typically applied to extension. This zero-padding strategy poses significant challenges in traffic classification with small samples. In this paper, we propose a method based on autoencoder (AE) and deep graph convolutional networks (ADGCN) for traffic classification for few-shot datasets. ADGCN first utilizes an AE to reconstruct the traffic. AE enables shorter traffic to learn abstract feature representations from longer traffic of the same class to replace zeros, mitigating the adverse effects of zero-padding. The reconstructed traffic is then classified using GCNII, a deep GCN model that addresses the challenge of insufficient data samples. ADGCN is an end-to-end traffic classification method applicable to various scenarios. According to experimental results, ADGCN can achieve a classification accuracy improvement of 3.5 to 24% compared to existing state-of-the-art methods. The code is available at https://github.com/han20011019/ADGCN.

Keywords Traffic classification, Autoencoder, Graph convolutional networks, Few-shot

Traffic classification is a significant technique in the field of computer networking, primarily functioning to categorize traffic into different classes based on various standards. This facilitates network management, ensures network security, and optimizes network architecture. It is important to many applications, such as quality of service (QoS) control, pricing, resource usage planning, malware detection, and intrusion detection[1]. Consequently, over the past two decades, the networking industry and research community have dedicated considerable effort to studying this technology. Numerous methods have been proposed, resulting in substantial advancements and successes. However, the continuous expansion of Internet and mobile technologies are creating a dynamic environment where new applications and services emerge every day, and the existing ones are constantly evolving[2]. For instance, countries around the world are now placing great emphasis on network security, and encrypted communication has become increasingly prevalent. This makes traffic classification challenging, as it is necessary to ensure normal service in the presence of encrypted traffic. Therefore, in the rapidly changing network environment, researching new traffic classification methods is essential.

Over time, traffic classification techniques have continuously evolved. The earliest forms of traffic classification were based on port numbers, leveraging the fact that the same protocols or applications typically used the same port numbers. However, this straightforward rule-based approach is easily evaded, resulting in a consistent decline in accuracy. The next generation of traffic classifiers, relying on payload or data packet inspection (DPI)[3,4]. This method involves directly analyzing the payload of data packets to extract specific keywords. Unfortunately, it is only applicable to unencrypted traffic and incurs significant computational cost.

[1]Information Security Research Institute, Beijing Electronic Science and Technology Institute, Beijing 100070, China. [2]Department of Cryptography Science and Technology, Beijing Electronic Science and Technology Institute, Beijing 100070, China. [3]School of Cyberspace Security, Beijing University of Posts and Telecommunications, Beijing 100876, China. ✉email: hseason1019@gmail.com

Subsequently, some scholars[2,5–8] extracted statistical features and used classical machine learning algorithms to achieve traffic classification. This approach heavily relies on expert-designed features, resulting in poor generalization capabilities and susceptibility to the influence of unreliable data. Deep learning, on the other hand, does not require domain experts to design specific features. It can automatically learn representations through training. This allows deep learning to capture the nonlinear relationships between raw data and corresponding outputs, forming an end-to-end model. This characteristic has made deep learning a highly popular method in recent years.

In essence, raw traffic is a form of sequential data. Kim et al.[9] and Song et al.[10] utilized the capability of recurrent neural networks (RNNs) in handling sequential data and their flexible input and output lengths, incorporating RNNs as foundational components in their own classification methods. However, RNN also has certain issues, such as vanishing and exploding gradients, making them difficult to use for tasks that involve long-term dependencies. Long short-term memory (LSTM), as a special type of RNN, can effectively address the challenges faced by traditional RNN. Hwang et al.[11] and Thapa et al.[12] used LSTM for traffic classification tasks. Wang et al.[13] made a novel attempt by transforming one-dimensional traffic data, which is sequential, into two-dimensional images, and then applying convolutional neural networks (CNNs) for learning and classification. Lopez-Martin et al.[14] combined CNN and RNN for traffic classification to leverage the advantages of both.

The aforementioned deep learning-based traffic classification methods all focus on classifying traffic based on its inherent features, without considering the relationships among traffic. Consequently, many scholars[15–18] have attempted to apply graph convolutional networks (GCNs) to traffic classification. GCN leverages the topological structure of graphs to directly perform learning on the graph, aggregating information from neighbors to enhance its own feature representations. However, in practical classification applications, GCN is typically set up with two layers, meaning it only incorporates neighbors within a distance of two from the node itself, i.e., the node's neighbors and its neighbors' neighbors. The primary reason is that deeper GCN layers can lead to the over-smoothing problem, which causes a sharp decline in accuracy. When training and classifying with large-scale traffic datasets, a two-layer GCN is sufficient as the large data volume makes it unnecessary to explore deeper relationships among the data. However, when dealing with few-shot traffic datasets, classification requires the use of deeper GCN to uncover deeper relationships. Nonetheless, simply increasing the number of GCN layers is not feasible due to the over-smoothing problem. Additionally, our experiments have found that in order to use GCN for traffic classification, it is necessary to construct a traffic graph. However, during the graph construction process, it is typically required to standardize the length of each traffic data instance. Although the standardized lengths vary widely, when the original traffic data is too short, zero-padding is commonly used to compensate. In few-shot scenarios, some part of traffic across different types contains a significant portion of zeros due to padding. This is highly detrimental to classification.

In this paper, we propose a method based on autoencoder and deep graph convolutional networks (ADGCN) for traffic classification for few-shot datasets. ADGCN is capable of addressing the aforementioned issues. Specifically, we have designed an end-to-end traffic classification model that takes PCAP files as input and outputs the category of traffic at the output end. Moreover, the model only requires a small amount of samples during training. ADGCN comprises seven steps in total. The first three steps involve preprocessing the raw traffic, including data cleaning, data trimming, and data integration. The fourth step is random sampling, which is used to create the few-shot dataset required for experiments. This ensures that each experimental dataset is randomly composed, thereby enhancing the reliability of the experimental results. The fifth step involves data reconstruction with an autoencoder (AE)[19]. This step is aimed at mitigating the adverse effects of zero-padding. In an abstract sense, the AE allows shorter traffic to learn abstract feature representations from longer traffic of the same class to replace zeros. The sixth step involves using the K-nearest neighbors (KNN) algorithm to construct the traffic graph. We use the heat kernel[20] to calculate the similarity among traffic. The seventh step employs the GCNII[21] model for traffic classification. GCNII is a deep GCN model that leverages deep learning to compensate for the lack of sufficient data in few-shot datasets.

The main contributions of this paper are summarized as follows:

(1) We cleverly leverage the characteristics of AE to reconstruct few-shot datasets, generating denoised and feature-optimized traffic feature representations. Specifically, AE enables shorter traffic sequences to learn abstract feature representations from longer sequences of the same type, replacing zeros and mitigating the adverse effects caused by zero-padding when standardizing traffic lengths for classification.
(2) We are the first to apply the GCNII model to traffic classification, breaking away from the conventional two-layer GCN-based approach. GCNII enables the exploration of deeper relationships between traffic flows, addressing the limitations posed by insufficient few-shot data.
(3) We propose a novel traffic classification method called ADGCN, which enables end-to-end traffic classification on small-sample datasets. We conducted partial sampling and a series of experiments on two public datasets, including accuracy experiments, ablation studies, and robustness tests. The experimental results demonstrate that ADGCN achieves an accuracy improvement of up to 3.5–24% compared to existing state-of-the-art methods. Additionally, it exhibits stronger robustness to dataset size and noise levels than advanced methods.

Related work

In this section, we introduce the most related work: various traffic classification methods and deep GCN.

Conventional traffic classification methods

Conventional traffic classification methods mainly include port-based and DPI-dependent approaches. Port-based traffic classification methods are highly susceptible to attacks due to the misuse of port information.

Subsequently, DPI-dependent traffic classification methods were introduced. Libprotoident[3] is a DPI library designed for application-layer protocol identification in traffic. Unlike many techniques that require capturing the entire packet payload, libprotoident only utilizes the first four bytes of the payload in each direction, the size of the first payload-carrying packet in each direction, and the TCP or UDP port numbers of the traffic. nDPI[4] is employed by ntop and nProbe for application-layer protocol detection, regardless of the port being used. This allows for the detection of known protocols on non-standard ports as well as mismatched protocols. Although DPI has achieved success and is widely used in some industry products, recent research indicates that DPI faces significant challenges due to encrypted traffic, which restricts access to raw data[1].

Traditional machine learning-based methods

Unlike DPI, traditional machine learning-based methods rely on statistical features, enabling them to handle encrypted traffic[2,5–8]. First, researchers design traffic features (e.g., the number of packets, minimum/maximum packet size) based on specific classification requirements (e.g., protocol/traffic type). These features are then fed into various classifiers based on machine learning models, including decision trees (DT)[22], k-nearest neighbors (kNN)[23], and support vector machines (SVM)[24], for classification. These methods break down the overall classification task into multiple sub-problems (e.g., feature derivation, machine learning model evaluation) and address them individually. However, simply combining optimal sub-solutions may not yield a globally optimal solution. Additionally, manual feature engineering has poor generalizability when dealing with different classification requirements[1].

Deep learning-based methods

Compared to traditional machine learning methods, deep learning-based approaches offer two major advantages. On the one hand, deep learning methods employ an end-to-end strategy, which makes it easier to achieve a globally optimal solution. On the other hand, neural networks can directly extract discriminative features from raw inputs, typically the original bytes of traffic. This feature extraction process is automated, significantly reducing the need for manual intervention.

Many deep neural network models have been applied to traffic classification[9–14], achieving promising results. Moreover, when GCN is used for traffic classification, it not only considers the intrinsic characteristics of the traffic but also takes into account the relationships among the traffic. Sun et al.[15] combined AE with GCN for classification. Initially, all raw traffic is standardized to a length of 900 bytes. Then, the KNN algorithm is employed to construct the traffic graph. Finally, traffic classification is performed using a combination of AE and GCN. The dimensionality of the feature representation output by each layer of GCN is unified with the dimensionality encoded by AE. Then, the outputs of both AE and GCN are combined in a certain proportion to serve as the learned feature representation for each layer of the model. Diao et al.[16] proposed a novel graph construction method in which, instead of standardizing all traffic to a fixed length, a fixed number of packets are selected from each traffic for graph construction. Since the length of data packets can vary up to a maximum value, known as the maximum transmission unit (MTU), the range from 0 to MTU is divided evenly into N intervals, creating N nodes, each representing a packet length interval. Following the temporal sequence of the traffic, the node corresponding to the length of the preceding data packet is connected to the node corresponding to the length of the subsequent data packet. This completes the basic structure of the graph. Then, one-hot encoding is employed to map the length of the data packet to the node features. Unlike the previous method, each traffic is represented as its own graph. Finally, GCN is used for classification.

Diao et al.[16] took a fixed number of packets from each traffic, which avoids the problem of zero-padding and allows variable traffic lengths. However, this method cannot be applied to traffic with a low number of packets, limiting its scope of use. Sun et al.[15]employed a standard approach to traffic trimming, normalizing all traffic to a uniform length. Traffic that falls short of the required length is padded with zeros. In few-shot scenarios, this zero-padding artificially extends the length of traffic and may negatively impact classification performance. Moreover, due to the over-smoothing problem, existing methods predominantly utilize two-layer GCN. In small-sample scenarios, these methods fail to capture deep relationships between data, resulting in suboptimal classification performance.

Pre-training models-based methods

Transformers excel in parallelization, modeling long-range dependencies, and adaptability across tasks, making them highly efficient and versatile for various applications. To effectively utilize unlabeled data, several traffic classification pretraining models based on Transformers have been proposed. Inspired by BERT's pretraining method in natural language processing, PERT[25] and ET-BERT[26] tokenize raw traffic bytes, apply masked language modeling to learn traffic representations, and fine-tune the models for downstream tasks. However, Transformer-based models face challenges in computational and memory efficiency due to the quadratic complexity of their core self-attention mechanism.

Mamba is designed for high efficiency, leveraging advanced architecture to optimize feature extraction, reduce computational overhead, and enhance performance across diverse tasks. Wang et al.[27] were the first to apply it to traffic classification, achieving excellent results. Not only does it exhibit very high classification accuracy, but it also outperforms existing methods in terms of computational complexity and efficiency. However, methods based on pre-trained models often require large amounts of raw data, and their performance may degrade in small-sample scenarios.

Deep GCN

Over-smoothing is one of the key issues which limit the performance of GCN as the number of layers increases. Many researchers have investigated this issue. Chen et al.[21] proposed the GCNII model, which introduces two new techniques: initial residual connections and Identity mapping.

Initial residual refers to the addition of the initial representation in each layer, ensuring that the node's intrinsic features are not diluted as the number of layers increases, thereby preventing over-smoothing. Identity mapping involves adding an identity matrix to the weight matrix. Bo et al.[20] used the data feature representations learned by an AE to modify the output of each layer of the GCN. Specifically, they aligned the output dimension of the AE encoder with the output dimension of each GCN layer, and then combined them in a certain proportion. Zhou et al.[28] proposed two indexes to measure over-smoothing: Gins and RGroup. Gins measures the relationship between the input and output, while RGroup measures the ratio of inter-class distance to intra-class distance for each category. Both indexes are better when larger. These indexes can be added to the loss function. In this paper, we apply the GCNII model to traffic classification, leveraging deep GCN to address the issue of insufficient sample size in the dataset.

Methodology

This section provides a detailed introduction to ADGCN. Figure 1 illustrates the overall workflow of the method, which consists of seven steps from acquiring raw traffic data from the dataset to the final classification of traffic. The first three steps of ADGCN involve preprocessing the raw traffic dataset using USTC-TK2016, a tool specifically designed for handling PCAP files, as published by Wang et al.[13]. After these three steps, we obtain binary data that is 784 bytes in length, non-redundant, and non-zero. The fourth step involves randomly sampling the preprocessed dataset to obtain a small sample dataset necessary for the experiment. Unlike existing deep learning-based traffic classification methods that directly input pre-processed datasets into deep neural network frameworks, ADGCN first performs data reconstruction on the pre-processed datasets. In Step 5, we ingeniously leverage the characteristics of the AE to reconstruct few-shot datasets, generating traffic feature representations that are denoised and feature-optimized. Specifically, AE enables shorter traffic sequences to learn abstract feature representations from longer sequences of the same type, replacing zero-padding. This reduces the adverse effects of zero-padding on traffic classification when unifying traffic lengths. Step 6 involves converting each piece of traffic data into a graph representation. Step 7 is the final classification phase. To address the issue that a two-layer GCN struggles to explore deep inter-data relationships in few-shot datasets, leading to low classification accuracy, we are the first to introduce GCNII into the traffic classification task in this paper. In the following sections, we will provide a detailed introduction to each step.

Step 1 traffic split

Split granularity of network traffic includes: TCP connection, flow, session, service, and host[29]. Different split granularity leads to distinct traffic units. The USTC-TK2016 toolkit provides two types of split granularity: flow and session. These two types of split are also widely used in many studies. A flow refers to a group of packets arranged in time order that share the same quintuple (source IP, source port, destination IP, destination port, and transport-level protocol) over a period of time, as shown in Fig. 2. A session includes both directions of flows, i.e. the source and destination IP / port are interchangeable, as shown in Fig. 3. Once each packet is grouped according to the specified traffic split granularity, USTC-TK2016 provides two processing options for each packet itself: L7 and ALL. L7 refers to retaining only layer 7 of the OSI model, while ALL refers to retaining all layers. In this paper, all experimental data are processed using the Session + L7 method.

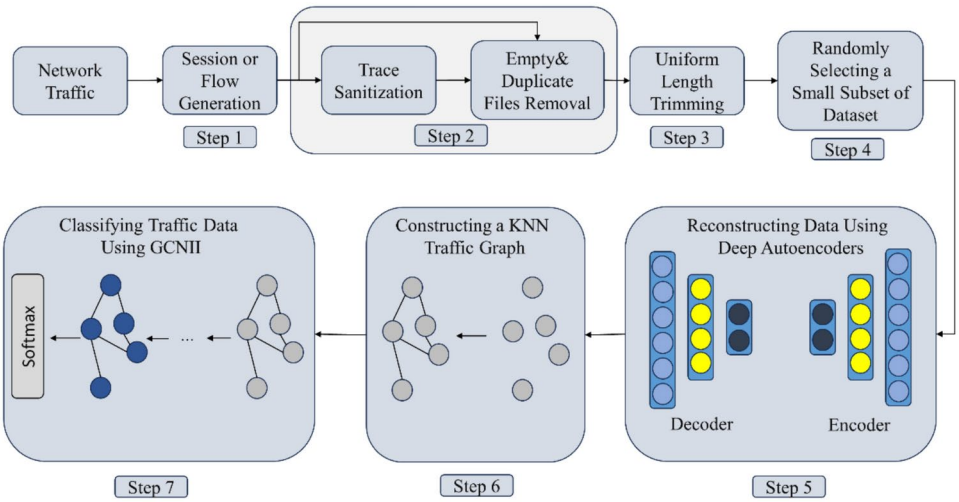

Fig. 1. The overall workflow of ADGCN.

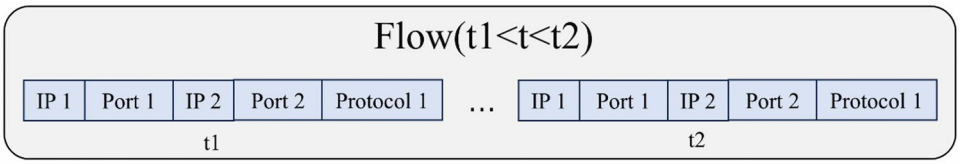

Fig. 2. Flow.

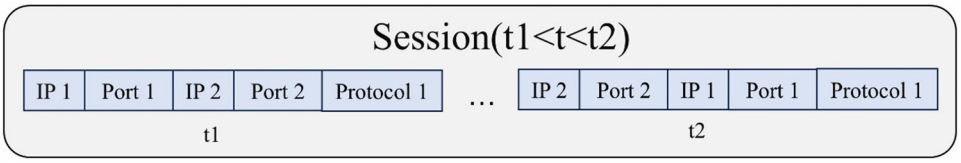

Fig. 3. Session.

Step 2 traffic clear

In this step, the process begins with trace sanitization, which involves randomizing the MAC address at the data link layer and the IP address at the IP layer, respectively. This is optional, for example, when all traffic is from the same network, the MAC and IP may no longer be the distinguishing information, and we don't need to perform it in this situation. In this paper, trace sanitization is not required because only application layer data is retained for each data packet. Next, the traffic files are cleaned, primarily by deleting empty files and duplicate files.

Step 3 uniform length trimming

After processing the data in the first two steps, we have obtained valid traffic data rather than discrete data packets from a real network environment. However, these data cannot be used for deep learning because they vary in length, so all traffic data must be transformed to a uniform length. Wang et al.[13] trimmed all data to 784 bytes, Sun et al.[15] trimmed all data to 900 bytes, and Xie et al.[30] experimented with trimming data to 40, 50, and 60 bytes respectively. In this paper, we trimmed all data to 784 bytes. For traffic longer than 784 bytes, the first 784 bytes are taken; for traffic shorter than 784 bytes, it is padded with zeros up to 784 bytes.

Step 4 random sampling

In recent years, researchers have achieved notable progress in traffic classification, as mentioned in the second part of this paper. This is because existing public traffic datasets are quite large, allowing models to achieve decent learning outcomes through prolonged training. In this paper, we propose a traffic classification method using small samples, which offers many advantages as discussed in the first part of this paper. In our approach, for all datasets, we randomly sample only 200 data points for each type of traffic data.

Step 5 reconstruction of the dataset

In the third step, all traffic data underwent a process of length standardization, where data with a length less than 784 bytes was padded with zeros. Although this approach is simple and efficient, it is not conducive to subsequent deep learning classification. This indiscriminate operation can lead to a long section of identical data (all zeros) at the end of different types of traffic data, which is detrimental to classification. Additionally, some traffic data naturally have very small lengths, such as packets that send control instructions. As illustrated in Fig. 4, for ease of visualization, each byte of traffic data is converted into an integer value between 0 and 255, and then the 784-byte data is transformed into a 28*28 matrix of integers. These integers are then converted to grayscale values to display the matrix as an image, where the black areas represent values of 0. These packets with very short lengths appear across different categories of data, and padding them with zeros to 784 bytes complicates the classification process further.

To address the issues mentioned above, this paper proposes a data reconstruction method using AE[19] to represent the data. The basic principle of the AE is illustrated in Fig. 5. An AE comprises two parts: an encoder and a decoder. The encoder compresses the original data $\mathbf{X}$ into $\mathbf{H}$ as an abstract feature representation for $\mathbf{X}$. The decoder uses this feature vector $\mathbf{H}$ to generate reconstructed data $\hat{\mathbf{X}}$. The loss function measures the difference between the original data $\mathbf{X}$ and the reconstructed data $\hat{\mathbf{X}}$.

Feature vector $\mathbf{H}$ can be generated by Eq. (1):

$$\mathbf{H}_e^{(m)} = \sigma(\mathbf{W}_e^{(m)}\mathbf{H}_e^{(m-1)} + \mathbf{b}_e^{(m)}) \tag{1}$$

Assuming the encoder consists of M layers, m in Eq. (1) denotes the m-th layer of the encoder, where $1 \leq m \leq M$. e means it is the variable in the encoder. $\mathbf{H}_e^{(m)}$ denotes the feature representation learned through the m-th layer of the encoder. $\mathbf{W}_e^{(m)}$ and $\mathbf{b}_e^{(m)}$ denote the weight matrix and biases in the m-th layer of the

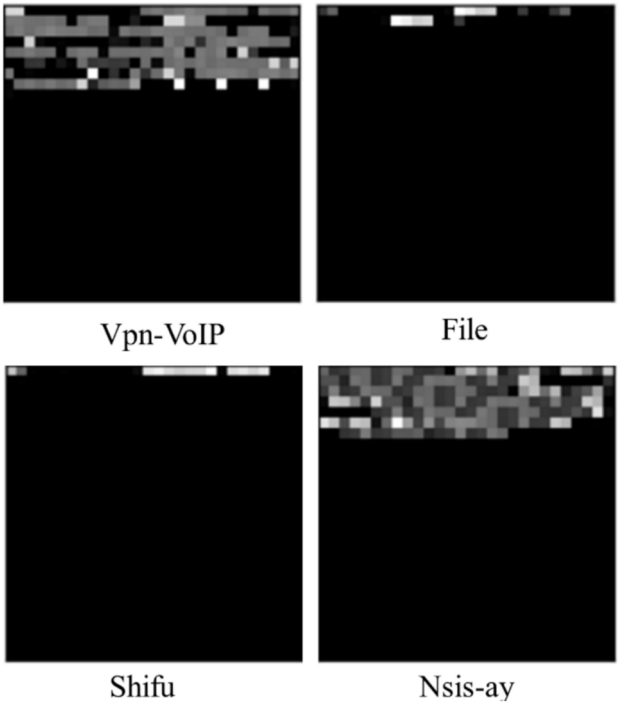

Fig. 4. Part of the traffic after zero-padding.

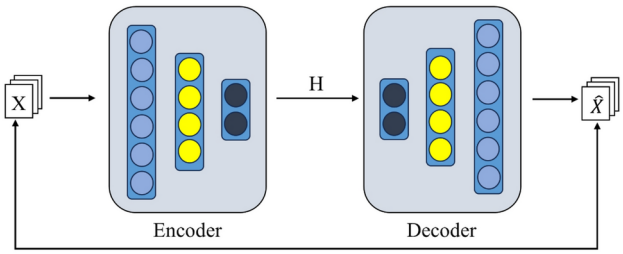

Fig. 5. Autoencoder.

encoder, respectively. σ denotes the activation function of the full connection layer such as ReLU[31] or Sigmoid function. Additionally, we define $\mathbf{H}_e^0$ as the original data $\mathbf{X}$ and $\mathrm{H}_e^{(M)}$ as the feature vector $\mathbf{H}$.

The reconstructed data $\hat{\mathbf{X}}$ can be obtained from Eq. (2).

$$\mathbf{H}_d^{(m)} = \sigma(\mathbf{W}_d^{(m)}\mathbf{H}_d^{(m-1)} + \mathbf{b}_d^{(m)}) \tag{2}$$

In the Eq. (2), m denotes the m-th layer of the decoder, where $1 \leq m \leq M$, d means it is the variable in the decoder. $\mathbf{H}_d^{(m)}$ refers to the reconstructed data at the m-th layer of the decoder. $\mathbf{W}_d^{(m)}$ and $\mathbf{b}_d^{(m)}$ denote the weight matrix and biases of the m-th layer of the decoder, respectively. Additionally, we define $\mathbf{H}_d^0$ as the feature vector $\mathbf{H}$ and $\mathbf{H}_d^{(M)}$ as the reconstructed data $\hat{\mathbf{X}}$.

The loss function $\mathcal{L}_R$ can be derived from Eq. (3).

$$\mathcal{L}_R = \frac{1}{2N}\sum_{i=1}^{N}\|\mathbf{x}_i - \widehat{\mathbf{x}}_i\|_2^2 = \frac{1}{2N}\mathbf{X} - \widehat{\mathbf{x}}_F^2 \tag{3}$$

In Eq. (3), N represents the quantity of reconstructed data in the training set.

In this paper, we convert each traffic into integers ranging from 0 to 255, essentially transforming single traffic into 784 integers to accommodate AE for computation. $\mathbf{X}$ represents each traffic. We reconstruct each type of traffic data in the dataset separately, use 80% of the traffic to train AE, and then reconstruct the remaining 20% of the traffic. Specifically, we use 160 items of the traffic from each type as the training set, then use AE to

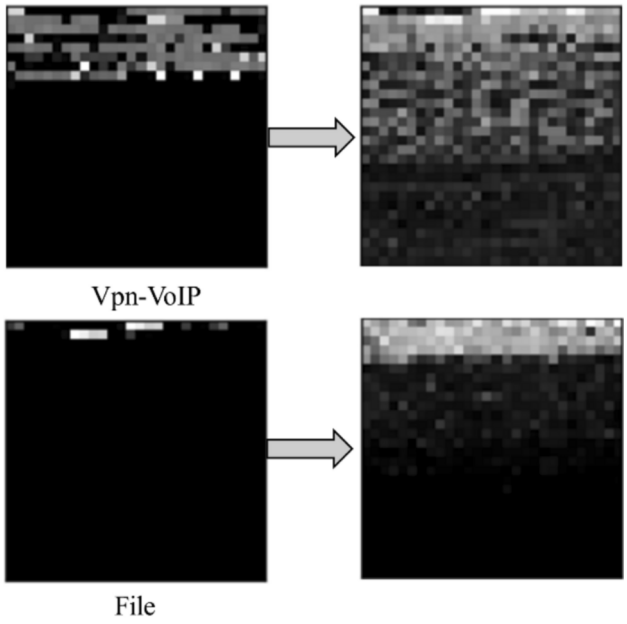

Fig. 6. The comparison between traffic reconstructed by AE and zero-padding.

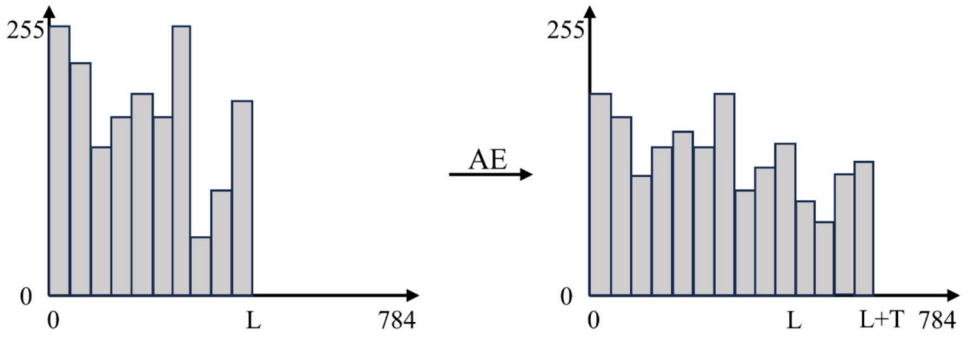

Fig. 7. The abstract representation of reconstructing traffic by AE.

reconstruct the remaining 40 items. Consequently, each type in the dataset contains only 40 items in subsequent work.

The effect of reconstructing data using AE is shown in Fig. 6. This reconstruction effect can be abstractly represented by Fig. 7.

AE can appropriately extend traffic that was only of length L before zero-padding to a length of $(L + T)$, and can also reduce the variance of individual bit values in the original data. This results in a smoother variation in the values of individual bits within the traffic data.

Step 6 construct the traffic graph

After the fifth step, 40 reconstructed traffic data samples were obtained for each type of traffic, resulting in a total of $n \times 40$ samples. Compared to other machine learning methods, GCN primarily takes into account the structural information among the data samples, based on the idea that the samples are interconnected and related. Since network traffic flows across the internet, which itself is a vast network graph, GCN is a suitable choice for traffic classification tasks. To use GCN for classification, a traffic graph must be constructed. In this paper, the KNN algorithm is used for constructing the graph. The basic principle of KNN is as follows: First, for each traffic item $\mathbf{X}$, the similarity between $\mathbf{X}$ and all other items is calculated. Then, the item $\mathbf{X}$ is connected to the k items with the highest similarity as graph nodes to form the graph's edges. In this paper, we use the Heat Kernel[20] to calculate the similarity matrix $\mathbf{S}$ between items. The similarity $\mathbf{S}_{ij}$ between item i and item j is obtained by Eq. (4).

$$\mathbf{S}_{ij} = e^{-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{t}} \tag{4}$$

In Eq. (4), $\|\mathbf{X}_i - \mathbf{X}_j\|^2$ represents the Euclidean distance between item i and item j, and t represents the time parameter in the heat conduction equation.

Step 7 traffic classification

After the sixth step, the traffic graph was obtained and represented as G. The basic process of using GCN for traffic classification is shown in Fig. 8. Suppose there are L layers of GCN. $\mathbf{M} \in R^{N\times C}$ is the matrix composed of all traffic items that need classification. N is the number of traffic items, and C is the number of features per item. In this paper, N is $n \times 40$, and C is 784. $\mathbf{Z}^{\ell} \in R^{N\times F}$ is the traffic representation matrix learned by the ℓ-th layer of GCN, with the desired dimension F, where $1 \leq \ell < L$. $\mathbf{Z}^{\ell}$ can be obtained by the given Eq. (5).

$$\mathbf{Z}^{(\ell)} = \phi\left(\tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-\frac{1}{2}}\mathbf{Z}^{(\ell-1)}\mathbf{W}^{(\ell-1)}\right) \tag{5}$$

In the Eq. (5), $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$, $\tilde{\mathbf{D}}_{ii} = \sum_j \tilde{\mathbf{A}}_{ij}$, where ϕ is the activation function, $\mathbf{W}^{(\ell-1)} \in R^{F\times H}$ is the weight matrix of the $(\ell-1)$-th layer, F is the traffic representation dimension obtained from the $(\ell-1)$-th layer, and H is the desired traffic representation dimension for the ℓ-th layer. $\mathbf{A} \in R^{N\times N}$ is the adjacency matrix of graph $\mathbf{G}$ and $\mathbf{I} \in R^{N\times N}$ is the identity matrix. $\tilde{\mathbf{D}} \in R^{N\times N}$ is a diagonal matrix representing the degree matrix. Additionally, $\mathbf{Z}^0 = \mathbf{M}$. The L-th layer of GCN is a multi-class classification layer with softmax activation function, which can be obtained using Eq. (6).

$$\mathbf{Z}^{L} = \mathrm{softmax}\left(\tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-\frac{1}{2}}\mathbf{Z}^{(L-1)}\mathbf{W}^{(L-1)}\right) \tag{6}$$

For a C-class classification problem, the loss function can be obtained using the Eq. (7):

$$\mathcal{L}_{\mathcal{C}} = -\sum_{l\in\mathcal{Y}_L}\sum_{c=1}^{C} Y_{lc}\ln \mathbf{Z}_{lc} \tag{7}$$

In the Eq. (7), y_L represents the set of node indices that have labels Y。

In the traffic classification task using GCN, while it is assumed that there are L layers of GCN, in practice, usually only two layers of GCN are used. This is because GCN can encounter the over-smoothing problem, which causes deeper layers of GCN to often have lower classification accuracy. This issue is contrary to the original design purpose of GCN, which aims to fully utilize the structural information between the data, but deeper layers of GCN actually lead to lower accuracy. To address this issue, many scholars have proposed their own methods in recent years, as mentioned in the second part of this paper. In this paper, we use the GCNII model proposed by Chen et al.[21]. This model introduces two simple techniques when calculating $\mathbf{Z}^{(\ell)}$, namely Initial Residual and Identity Mapping. $\mathbf{Z}^{(\ell)}$ can be calculated using Eq. (8).

$$\mathbf{Z}^{(\ell)} = \sigma\left(\left((1-\alpha_{\ell-1})\,\widetilde{\mathbf{P}}\mathbf{Z}^{(\ell-1)} + \alpha_{\ell-1}\mathbf{Z}^{(0)}\right)\left((1-\beta_{\ell-1})\,\mathbf{I}_n + \beta_{\ell-1}\mathbf{W}^{(\ell-1)}\right)\right) \tag{8}$$

In Eq. (8), $\tilde{\mathbf{P}} = \tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-\frac{1}{2}}$, where $\alpha_{\ell-1}$ and $\beta_{\ell-1}$ are two hyperparameters. $\beta_{\ell-1} = \log(\lambda/(\ell-1)+1)$, while $\alpha_{\ell-1}$ and λ are constants between 0.1 and 1. Initial residual refers to adding the initial representation in each layer so that the node's own features are not diluted as the number of layers increases, thus avoiding over-smoothing. Identity mapping involves adding an identity matrix in the weight matrix. This idea is inspired by ResNet[32], meaning that $\mathbf{P}\mathbf{Z}^{(\ell-1)} + \mathbf{Z}^0$ is directly mapped to the output, allowing for the use of regularization techniques to alleviate overfitting while retaining higher-order information.

Experiments

In Section "Experiment setup", the preparation for the experiment is introduced, including datasets, experimental environment, hyperparameter selection, and evaluation indexes. In Section "Contrast experiment", ADGCN is compared with five other methods, including both traditional machine learning methods and deep learning methods. In Section "Ablation experiment", to enhance the interpretability of the proposed method, ablation

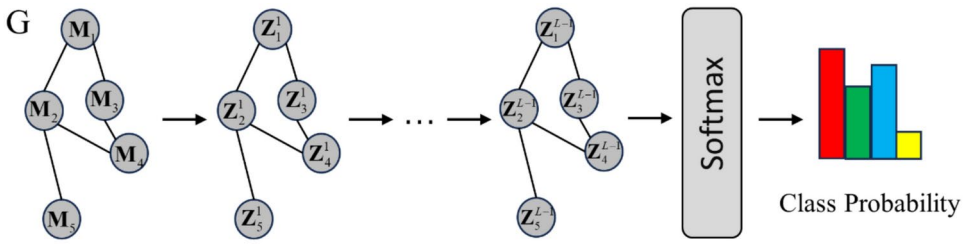

Fig. 8. L-layer GCN classification.

Traffic type	CTU num	Binary MD5	Process
Cridex	108–1	25b8631afeea279ac00b2da70fffe18a	Original
Geodo	119–2	306573e52008779a0801a25fafb18101	Part
Htbot	110–1	e515267ba19417974a63b51e4f7dd9e9	Original
Miuref	127–1	a41d395286deb113e17bd3f4b69ec182	Original
Neris	42,43	bf08e6b02e00d2bc6dd493e93e69872f.	Merged
Nsis-ay	53	eaf85db9898d3c9101fd5fcfa4ac80e4	original
Shifu	142–1	b9bc3f1b2aace824482c10ffa422f78b	Part
Tinba	150–1	e9718e38e35ca31c6bc0281cb4ecfae8	Part
Virut	54	85f9a5247afbe51e64794193f1dd72eb	Original
Zeus	116–2	8df6603d7cbc2fd5862b14377582d46	Original

Table 1. USTC-TFC2016 Part I (malware traffic).

Traffic type	Class	Traffic type	Class
BitTorrent	P2P	Outlook	Email/WebMail
Facetime	Voice/Video	Skype	Chat/IM
FTP	Data Transfer	SMB	Data transfer
Gmail	Email/WebMail	Weibo	Social NetWork
MySQL	Database	WorldOfWarcraft	Game

Table 2. USTC-TFC2016 Part II (normal traffic).

experiments were conducted on ADGCN. In Section "Robustness experiment", robustness tests were carried out on ADGCN to demonstrate its performance in some extreme environments.

Experiment setup

Datasets

To verify the reliability of ADGCN and enhance the credibility of the experimental results, all experiments in this paper are conducted using two public traffic datasets: USTC-TFC2016 and ISCX-VPN-NonVPN-2016. These datasets were both collected from real network environments and consist of raw traffic data. Detailed introductions to the two datasets are as follows:

USTC-TFC2016:

The dataset was established by Wang et al.[13] and consists of two parts, as shown in Tables 1 and 2. The first part includes malware traffic from 10 real network environments, obtained by CTU researchers from public websites between 2011 and 2015[33]. In some cases, a portion of larger-scale traffic was used, while smaller-scale traffic was merged with similar types. The second part comprises normal traffic from 10 real network environments, collected by the creators using IXIA BPS[34].

ISCX-VPN-NonVPN-2016:

The dataset was collected by Draper-Gil et al.[2] using Wireshark and tcpdump from a real network environment, where laboratory members created accounts and used services like Skype and Facebook, as shown in Table 3. The dataset comprises 7 categories of data, each with normal and VPN protocol-encapsulated data formats, leading to a total of 14 labels. However, since Wang et al.[35] and Xie et al.[30] both noted issues with the "Browser" and "VPN-Browser" data in the dataset, our experiments only used the remaining portion of the dataset, amounting to a total of 12 labels.

Experimental environment

All experiments were conducted on a laptop equipped with an Intel(R) Core(TM) i5-9300H @ 2.40 GHz CPU, 16.0 GB RAM, GTX 1650 GPU, and Windows 11 Home Edition OS. We used PyTorch as the deep learning software framework to implement our methods, with version 2.1.0 and Python version 3.11.5.

Hyper parameter selection

When using AE to reconstruct traffic data, the encoder's dimensions were set to 784-128-64-32, and the decoder's dimensions were set to 32-64-128-784. The original dimension of the traffic data is 784, while the hidden layers in the encoder contain 128 and 64 neurons, respectively. The learned feature representation dimension is 32. For reconstructing each type of traffic, 80% of the data was used for training, and 20% of the data was reconstructed using the trained AE for subsequent classification tasks. During training, the model ran for 10 epochs, using the Adam optimizer and the binary cross-entropy loss function.

When constructing the traffic graph using the KNN algorithm, the value of k was set to 10, meaning each traffic item was connected to its 10 most similar items based on similarity measures.

When training the model using the GCNII model, the parameter α was set to 0.2, and λ was set to 0.5. The model had 4 layers, a learning rate of 0.005, and a dropout rate of 0.6. The hidden layer dimension was set to 64.

Traffic type	Content
Browser	Firefox and Chrome
VPN-Browser	
Email	SMPTS, POP3S and IMAPS
VPN-Email	
Chat	ICQ, AIM, Skype, Facebook and Hangouts
VPN-Chat	
Streaming	Vimeo and Youtube
VPN-Streaming	
File transfer	Skype, FTPS and SFTP using Filezilla and an external service
VPN-File transfer	
VoIP	Facebook, Skype and Hangouts voice calls (1 h duration)
VPN-VoIP	
P2P	uTorrent and Transmission (Bittorrent)
VPN-P2P	

Table 3. ISCX-VPN-NonVPN-2016.

During training, the maximum number of epochs was set to 3000, with a patience value of 200 epochs. If the accuracy on the test set remained unchanged for 200 epochs, training was stopped.

Evaluation indexes
To compare the classification performance of ADGCN with other methods, we used four popular indexes: accuracy, precision, recall, and F1 score.

Accuracy can be obtained by Eq. (9).

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN} \tag{9}$$

Precision can be obtained by Eq. (10).

$$\text{Precision} = \frac{TP}{TP + FP} \tag{10}$$

Recall can be obtained by Eq. (11).

$$\text{Recall} = \frac{TP}{TP + FN} \tag{11}$$

F1score can be obtained by Eq. (12).

$$\text{F1} - Score = \frac{2Precision \cdot Recall}{Precision + Recall} \tag{12}$$

In the aforementioned equation, TP refers to the number of instances correctly classified as a particular category. FP refers to the number of instances incorrectly classified as that category. FN refers to the number of instances that should have been classified as a particular category but were classified as other categories. TN refers to the number of instances correctly classified as not being a particular category.

Contrast experiment

In this section, to validate the effectiveness of ADGCN, we compare it with five other methods: KNN[36], GCNII[21], CNN[13], SAM[30], and NetMamba[27]. KNN is a traditional machine learning-based method, while GCNII, CNN, and SAM are deep learning-based methods. NetMamba is a pre-learning-based method. For KNN, we set K to 5. For GCNII, the number of layers is set to 2, α is set to 0.1, λ is set to 0.5, the hidden layer dimension is set to 64, the learning rate is set to 0.005, and dropout is set to 0.6. For CNN, the number of convolutional layers is 1, the kernel size is 3×3, the input channel size is 1, the output channel size is 32, the number of fully connected layers is 1, and the learning rate is 0.005. For SAM, L is set to 50, and other parameters are kept at their defaults. For NetMamba, epochs for pre-training is set to 400, and other parameters are kept at their defaults.

In ADGCN, when using AE for data reconstruction, each class uses 160 traffic samples as the training set, and the trained AE is then used to reconstruct the remaining 40 traffic samples. When using GCNII for traffic classification, the 40 reconstructed traffic samples for each class are divided into 10 for training, 5 for validation, and 25 for testing. For KNN, 80% of the samples are used for training and 20% for testing. For CNN, the training set accounts for 80% and the testing set for 20%. In GCNII, the training set accounts for 50%, the validation set for 25% and the testing set for 25%. In SAM, the training set accounts for 50% and the testing set for 50%. In NetMamba, the training set accounts for 80%, the validation set for 10% and the testing set for 10%. All

	Accuracy	Precision	Recall	F1-score
KNN	63.75	81.62	63.75	64.4
CNN	79.52	79.76	80.18	79.72
GCNII	47.68	67.16	46.94	49.25
SAM	84.35	85.7	84.15	84.06
NetMamb	81.5	79.67	81.5	80.14
ADGCN	96.2	96.99	96.4	96.34

Table 4. Results of all methods on the USTC-TFC2016 dataset for 20-class classification.

	Accuracy	Precision	Recall	F1-score
KNN	71.5	80.47	71.5	71.66
CNN	88.75	89.03	89.2	88.93
GCNII	53.12	69.58	53.36	51.06
SAM	91.35	92.01	91.4	91.52
NetMamb	81	77.27	81	78.26
ADGCN	98.6	99.09	98	98.99

Table 5. Results of all methods on the USTC-TFC2016-MALWARE dataset for 10-class classification.

	Accuracy	Precision	Recall	F1-score
KNN	65.75	79.61	65.72	64.16
CNN	81.75	80.99	80.49	80.5
GCNII	56.82	79.77	53.24	58.07
SAM	85.45	85.64	85.1	85.07
NetMamb	81.5	82.78	81.5	81.59
ADGCN	98.8	98.41	97.99	97.96

Table 6. Results of all methods on the USTC-TFC2016-NORMAL dataset for 10-class classification.

	Accuracy	Precision	Recall	F1-score
KNN	54.58	62.36	54.58	57.04
CNN	70.42	75.39	69.35	69.05
GCNII	56.77	63.69	58.1	55.19
SAM	62.08	64.51	64.42	63.61
NetMamb	66.25	69.13	66.25	62.94
ADGCN	94.33	94.89	94	93.78

Table 7. Results of all methods on the ISCX-VPN-NonVPN-2016 dataset for 12-class classification.

experiments are conducted on datasets containing 200 samples per class. The reported experimental results are the average of 20 experiments.

The experimental results shown in Table 4 are obtained from 20-class classification on the USTC-TFC2016 dataset. ADGCN demonstrates a 11.85% improvement in accuracy compared to SAM, which performed relatively well.

The experimental results shown in Table 5 are obtained from 10-class classification on the malicious traffic portion of the USTC-TFC2016 dataset. ADGCN demonstrates a 7.25% improvement in accuracy compared to SAM, which performed relatively well.

The experimental results presented in Table 6 are derived from 10-class classification on the normal traffic portion of the USTC-TFC2016 dataset. ADGCN demonstrates a 13.35% improvement in accuracy compared to SAM, which performed relatively well.

The experimental results presented in Table 7 are derived from 12-class classification on the ISCX-VPN-NonVPN-2016 dataset. ADGCN demonstrates a 23.91% improvement in accuracy compared to CNN, which performed relatively well.

	Accuracy	Precision	Recall	F1-score
KNN	82.5	84.98	82.5	82.85
CNN	92.92	93.11	92.93	92.73
GCNII	86.87	87.62	86.2	86.2
SAM	90.04	89.8	89.83	89.76
NetMamb	94.17	94.65	94.17	94
ADGCN	97.67	97.31	97.06	97.02

Table 8. Results of all methods on the ISCX-VPN-NonVPN-2016-VPN dataset for 6-class classification.

	Accuracy	Precision	Recall	F1-score
KNN	55.42	59.93	55.42	56.87
CNN	72.08	82.24	72.08	69.97
GCNII	57.01	58.49	56.33	50.55
SAM	57.64	60.58	59.67	60.01
NetMamb	62.5	65.06	62.5	62.56
ADGCN	96.67	96.85	96.67	96.66

Table 9. Results of all methods on the ISCX-VPN-NonVPN-2016-NoVPN dataset for 6-class classification.

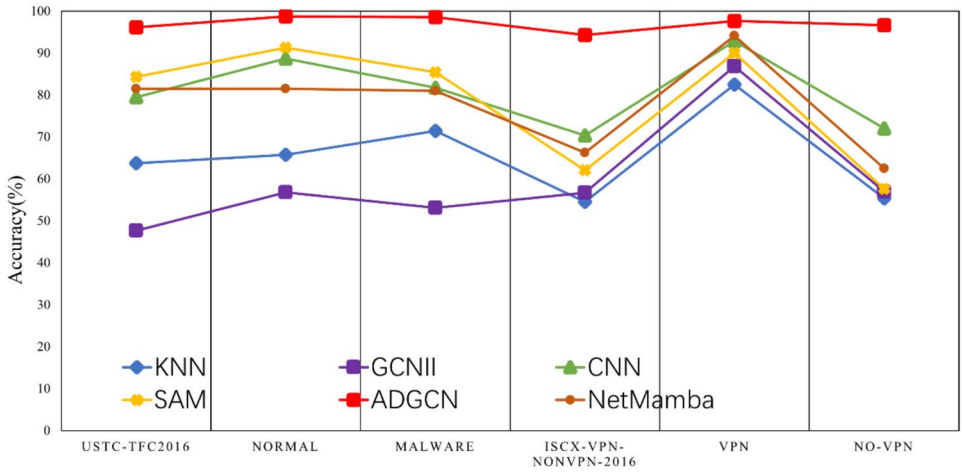

Fig. 9. The accuracy comparison of the six methods across different traffic classification scenarios. The vertical axis represents accuracy measured in percentage (%), while the horizontal axis denotes various traffic classification scenarios.

The experimental results shown in Table 8 are obtained from 6-class classification on the ISCX-VPN-NonVPN-2016 dataset, focusing on traffic encapsulated through VPN protocols. ADGCN demonstrates a 3.5% improvement in accuracy compared to NetMamb, which performed relatively well.

The experimental results shown in Table 9 are obtained from 6-class classification on the ISCX-VPN-NonVPN-2016 dataset, focusing on regular encrypted traffic. ADGCN demonstrates a 24.59% improvement in accuracy compared to CNN, which performed relatively well.

As shown in Fig. 9, we compare the accuracy of these methods across the six scenarios of traffic classification. We observe that ADGCN consistently maintains the highest accuracy across all six scenarios, with relatively stable results. The accuracy of the other five methods fluctuates significantly across different classification scenarios, and none of them consistently outperforms the others.

This clearly demonstrates that ADGCN is suitable for various traffic classification scenarios. In addition, it maintains high accuracy in the two most challenging scenarios (12-class classification on the ISCX-VPN-NonVPN-2016 dataset and 6-class classification on normal encrypted traffic in the ISCX-VPN-NonVPN-2016 dataset), with accuracies of 94.33% and 96.67%, respectively. This represents an improvement of 23.91% and 24.59% over the other five methods, respectively.

	USTC-TFC2016	MALWARE	NORMAL	ISCX-VPN-NonVPN-2016	VPN	NO-VPN
ADGCN-AE	47.68	53.12	56.82	56.77	86.87	57.01
ADGCN-Heat Kernel	70.85	87.09	81.83	79.33	94.52	77.67
ADGCN-GCNII	91.55	80.99	93.49	88.08	88.01	77.33
ADGCN	96.2	98.6	98.8	94.33	97.67	96.67

Table 10. the accuracy of the ablation experiments on ADGCN.

	ADGCN	SAM	CNN	KNN	NetMamb
100%	96.2	84.35	79.52	63.75	81.5
50%	93.6	79.01	73.48	58.47	73.5
25%	88.01	73.38	68.57	52.18	71.03

Table 11. The accuracy of dataset size robustness experiment on USTC-TFC2016 dataset.

	ADGCN	SAM	CNN	KNN	NetMamb
100%	94.33	62.08	70.42	54.58	66.25
50%	91.06	54.79	67.5	49.63	55.83
25%	86.75	41.71	57.5	41.74	50.76

Table 12. The accuracy of dataset size robustness experiment on ISCX-VPN-NonVPN-2016 dataset.

Ablation experiment

To explore the interpretability of ADGCN, in this section, we conducted ablation experiments on ADGCN to demonstrate the roles and contributions of each component in traffic classification. The results of the ablation experiments are shown in Table 10, where ADGCN-GCNII represents using two layers of GCN directly for classification. ADGCN-Heat Kernel indicates using the ordinary cosine similarity for computing the similarity among traffic when constructing the traffic graph with KNN. ADGCN-AE means we do not use AE for traffic reconstruction. Instead, we directly construct the graph, and then perform classification. From the results of the ablation experiments, it can be seen that using AE for traffic reconstruction is crucial, which also indicates that zero-padding indeed has a significant impact on classification. Although Heat Kernel and GCNII have different effects on classification accuracy in different scenarios, both contribute to accuracy improvement. In some scenarios, there is also an improvement of over 20%. This indicates that deeper GCNs can indeed exploit more information to compensate for the lack of data volume in small samples. Additionally, employing a better method for calculating the similarity between traffic can facilitate clustering of more similar traffic together.

Robustness experiment

To demonstrate the stability of ADGCN, in this section, we conducted robustness experiments on ADGCN, KNN, CNN, SAM and NetMamba to compare their performance under stringent conditions. As GCNII has lower accuracy, we did not include it in the robustness experiments in this section. In Sect. 4.3.1, we conducted experiments on dataset sample quantity robustness. In Sect. 4.3.2, we conducted experiments on the robustness of original data noise levels.

Robustness of datasets size

In practical applications, it is possible that the existing data may not meet the requirement of having 200 samples per class as set in this paper, meaning there may be fewer samples available. In this section, we conducted experiments on five methods with dataset sizes reduced to 50% and 25% respectively. The experimental accuracies are shown in Tables 11 and 12. Table 11 presents the accuracy of 20-class classification on the USTC-TFC2016 dataset, while Table 12 presents the accuracy of 12-class classification on the ISCX-VPN-NonVPN-2016 dataset. From the tables, we can observe that all five methods exhibit decent stability in this scenario, but the stability of ADGCN is slightly higher than the other four methods. When the sample size is reduced to 50% of the original, the accuracy decreased by 2.6% and 3.27% respectively. When the sample size is further reduced to 25%, the accuracy decreased by 8.19% and 7.58% respectively.

Robustness of noise level

In the process of collecting raw traffic data, noise may inadvertently be introduced, which is quite common in practical applications. To address this, we introduced Gaussian noise into the original dataset to simulate this scenario and compare the robustness of the five methods under different levels of noise. In this section, we conducted experiments by adding Gaussian noise with mean 0 and variances of 0.2, 0.4, and 0.6 to the original dataset, respectively.

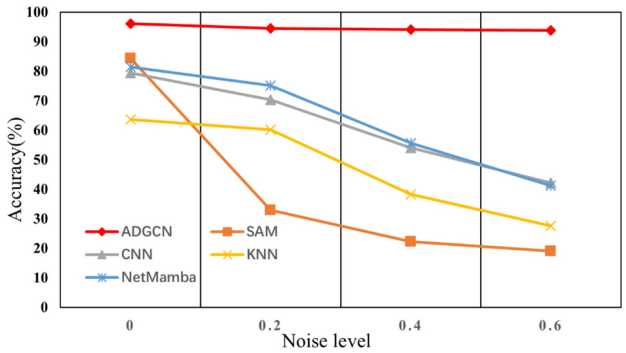

Fig. 10. The accuracy of dataset noise level robustness experiment on USTC-TFC2016 dataset.

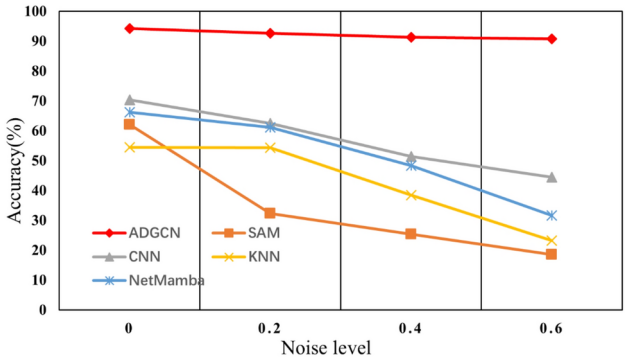

Fig. 11. The accuracy of dataset noise level robustness experiment on ISCX-VPN-NonVPN-2016 dataset.

The experimental accuracies are shown in Figs. 10, 11 respectively. Figure 10 displays the accuracy of 20-class classification on the USTC-TFC2016 dataset, while Fig. 11 presents the accuracy of 12-class classification on the ISCX-VPN-NonVPN-2016 dataset. From the figures, it is evident that ADGCN exhibits greater stability in the presence of noise compared to the other four methods, contrasting with its performance in robustness to dataset sample size. This is mainly attributed to the use of AE for traffic reconstruction in ADGCN, as AE is known for its denoising capabilities.

Conclusion

Traffic classification plays a crucial role in various fields such as network management and security. Through effective traffic classification, network administrators can identify and prioritize critical data, ensuring the rational allocation of network resources. Additionally, traffic classification aids in identifying and thwarting potential network attacks, thereby enhancing network security. In this paper, we propose ADGCN, a novel traffic classification method. ADGCN effectively combines AE and GCNII to achieve traffic classification on few-shot datasets. ADGCN ingeniously leverages the characteristics of AE by using them for traffic reconstruction, thereby mitigating the adverse effects of zero padding. Subsequently, GCNII is employed for traffic classification. GCNII, a deep GCN, compensates for the deficiency of few-shot datasets by exploiting the advantages of deep learning techniques. ADGCN demonstrates versatility in traffic classification across various scenarios. In this paper, we conducted experiments on encrypted malicious traffic classification, encrypted regular traffic classification, and encrypted traffic classification under VPN scenarios, all of which presented decent results. ADGCN exhibits a minimum improvement of approximately 3.5% in accuracy compared to existing methods, observed in the scenario of six-class classification of encrypted traffic under VPN conditions. Its maximum improvement, approximately 24% in accuracy, was observed in the classification of regular encrypted traffic on the ISCX-VPN-NonVPN-2016 dataset. The drawback lies in the use of AE for data reconstruction, which requires a majority of longer-length traffic instances within a class to learn feature representations from other instances instead of relying on zero-padding for shorter-length traffic. If a class predominantly consists of shorter-length traffic, it becomes challenging to learn new feature representations to replace zeros after reconstruction. In future work, we aim to explore novel methods to address this dilemma.

Data availability

The datasets generated and/or analysed during the current study are available in the [ADGCN] repository, [https://github.com/han20011019/ADGCN/tree/main/ADGCN/data]. The code is available at https://github.com/han20011019/ADGCN.

Received: 16 August 2024; Accepted: 12 March 2025
Published online: 15 March 2025

References

1. Rezaei, S. & Liu, X. Deep learning for encrypted traffic classification: An ove-rview. *IEEE Commun. Mag.* **57**(5), 76–81. https://doi.org/10.1109/mcom.2019.1800819 (2019).
2. Gil, G. D., Lashkari, A. H., Mamun, M., et al. Characterization of encrypted and VPN traffic using time-related features[C]. In *Proceedings of the 2nd International Conference on Information Systems Security and Privacy (ICISSP 2016). SciTePress*, 407–414. https://doi.org/10.5220/0005740704070414 (2016).
3. Alcock, S. & Nelson, R. Libprotoident: Traffic classification using lightweight packet inspection[R]. Technical Report, University of Waikato (2012).
4. Deri, L., Martinelli, M., Bujlow, T., et al. ndpi: Open-source high-speed deep packet inspection. In *2014 International Wireless Communications and Mobile Computing Conference (IWCMC)*. IEEE, 617–622. https://doi.org/10.1109/iwcmc.2014.6906427 (2014).
5. Van Ede, T., Bortolameotti, R., Continella, A., et al. Flowprint: Semi-supervised mobile-app fingerprinting on encrypted network traffic. In Network and Distributed System Security Symposium. https://doi.org/10.14722/ndss.2020.24412 (2020).
6. Panchenko, A., Lanze, F., Pennekamp, J., et al. Website fingerprinting at internet scale. NDSS. https://doi.org/10.14722/ndss.2016.23477 (2016).
7. Yang, J., Narantuya, J. & Lim, H. Bayesian neural network based encrypted traffic classification using initial handshake packets. In *2019 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks–Supplemental Volume (DSN-S)*. IEEE, 19–20. https://doi.org/10.1109/dsn-s.2019.00015 (2019).
8. Shen, M. et al. Optimizing feature selection for efficient encrypted traffic classification: A systematic approach. *IEEE Netw.* **34**(4), 20–27. https://doi.org/10.1109/mnet.011.1900366 (2020).
9. Kim, K. et al. Deep RNN-based network traffic classification scheme in edge computing system. *Comput. Sci. Inf. Syst.* **19**(1), 165–184. https://doi.org/10.2298/csis200424038k (2022).
10. Song, Z. et al. RNN: An incremental and interpretable recurrent neural network for encrypted traffic classification. *IEEE Trans. Dependable Secure Comput.* https://doi.org/10.1109/tdsc.2023.3245411 (2023).
11. Hwang, R. H. et al. An LSTM-based deep learning approach for classifying malicious traffic at the packet level. *Appl. Sci.* **9**(16), 3414. https://doi.org/10.3390/app9163414 (2019).
12. Thapa, K. N. K. & Duraipandian, N. Malicious traffic classification using long short-term memory (LSTM) model. *Wirel. Pers. Commun.* **119**(3), 2707–2724. https://doi.org/10.1007/s11277-021-08359-6 (2021).
13. Wang, W., Zhu, M., Zeng, X., et al. Malware traffic classification using convolutional neural network for representation learning. In *2017 International Conference on Information Networking (ICOIN)*. IEEE, pp. 712–717. https://doi.org/10.1109/icoin.2017.7899588 (2017).
14. Lopez-Martin, M. et al. Network traffic classifier with convolutional and recurrent neural networks for Internet of Things. *IEEE Access* **5**, 18042–18050. https://doi.org/10.1109/access.2017.2747560 (2017).
15. Sun, B., Yang, W., Yan, M., et al. An encrypted traffic classification method combining graph convolutional network and autoencoder. In *2020 IEEE 39th International Performance Computing and Communications Conference (IPCCC)*. IEEE, 1–8. https://doi.org/10.1109/ipccc50635.2020.9391542(2020).
16. Diao, Z. et al. EC-GCN: A encrypted traffic classification framework based on multi-scale graph convolution networks. *Comput. Netw.* **224**, 109614. https://doi.org/10.1016/j.comnet.2023.109614 (2023).
17. Mo, S., Wang, Y., Xiao, D., et al. Encrypted traffic classification using graph convolutional networks. Advanced Data Mining and Applications: 16th International Conference, ADMA 2020, Foshan, China, November 12–14, 2020, Proceedings 16. Springer International Publishing, pp. 207–219. https://doi.org/10.1007/978-3-030-65390-3_17 (2020).
18. Pang, B., Fu, Y., Ren, S. et al. CGNN: traffic classification with graph neural network. arXiv preprint arXiv:2110.09726. https://doi.org/10.48550/arXiv.2110.09726 (2021).
19. Hinton, G. E. & Salakhutdinov, R. R. Reducing the dimensionality of data with neural networks. *Science* **313**(5786), 504–507. https://doi.org/10.1126/science.1127647 (2006).
20. Bo, D., Wang, X., Shi, C., et al. Structural deep clustering network. *Proceedings of the Web Conference 2020*, pp. 1400–1410. https://doi.org/10.1145/3366423.3380214 (2020).
21. Chen, M., Wei, Z., Huang, Z., et al. Simple and deep graph convolutional networks. *International Conference on Machine Learning*. PMLR, 1725–1735 (2020).
22. Quinlan, J. R. Induction of decision trees. *Mach. Learn.* **1**, 81–106. https://doi.org/10.1007/BF00116251 (1986).
23. Cover, T. & Hart, P. Nearest neighbor pattern classification. *IEEE Trans. Inf. Theory* **13**(1), 21–27. https://doi.org/10.1109/TIT.1967.1053964 (1967).
24. Cortes, C. & Vapnik, V. Support vector machine. *Mach. Learn.* **20**(3), 273–297 (1995).
25. He, H. Y., Yang, Z. G. & Chen, X. N. Pert: Payload encoding representation from transformer for encrypted traffic classification. In *2020 ITU Kaleidoscope: Industry-Driven Digital Transformation (ITU K)*. IEEE, pp. 1–8. https://doi.org/10.23919/ITUK50268.2020.9303204 (2020).
26. Lin, X., Xiong, G., Gou, G., et al. Et-bert: A contextualized datagram representation with pre-training transformers for encrypted traffic classification. *Proceedings of the ACM Web Conference 2022*. 633–642 (2022).
27. Wang, T., Xie, X., Wang, W., et al. NetMamba: Efficient network traffic classification via pre-training unidirectional Mamba. arXiv preprint arXiv:2405.11449. https://doi.org/10.1145/3485447.3512217 (2024).
28. Zhou, K. et al. Towards deeper graph neural networks with differentiable group normalization. *Adv. Neural Inf. Process. Syst.* **33**, 4917–4928 (2020).
29. Dainotti, A., Pescape, A. & Claffy, K. C. Issues and future directions in traffic classification. *IEEE Netw.* **26**(1), 35–40. https://doi.org/10.1109/mnet.2012.6135854 (2012).
30. Xie, G., Li, Q. & Jiang, Y. Self-attentive deep learning method for online traffic classification and its interpretability. *Comput. Netw.* **196**, 108267. https://doi.org/10.1016/j.comnet.2021.108267 (2021).
31. Nair, V. & Hinton, G. E. Rectified linear units improve restricted boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning (ICML-10)*, 807–814 (2010).
32. He, K., Zhang, X., Ren, S., et al. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770–778. https://doi.org/10.1109/cvpr.2016.90 (2016).
33. CTU University, The Stratosphere IPS Project Dataset, https://stratosphereips.org/category/dataset.html (2024).
34. Ixia Corporation, Ixia Breakpoint Overview and Specifications, https://www.ixiacom.com/products/breakingpoint (2024).
35. Wang, W., Zhu, M., Wang, J., et al. End-to-end encrypted traffic classification with one-dimensional convolution neural networks. In *2017 IEEE International Conference on Intelligence and Security Informatics (ISI)*. IEEE, pp. 43–48. https://doi.org/10.1109/isi.2017.8004872 (2017).
36. Cover, T. & Hart, P. Nearest neighbor pattern classification. *IEEE Trans. Inform. Theory* **13**(1), 21–27. https://doi.org/10.1109/tit.1967.1053964 (1967).

Acknowledgements
This work is supported by the National Key Research and Development Program of China [Grant Number 2022YFB3104402]; the Fundamental Research Funds for the Central Universities [Grant Number 3282023035].

Author contributions
Shengwei Xu: Resources, supervision, project administration, Funding acquisition. Jijie Han: Conceptualization, methodology, software, investigation, writing—original draft. Yilong Liu: Conceptualization, methodology, writing—review and editing. Haoran Liu: validation, data curation, visualization. Yijie Bai: formal analysis, data curation, visualization.

Declarations

Competing interests
The authors declare no competing interests.

Additional information
Correspondence and requests for materials should be addressed to J.H.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

© The Author(s) 2025